Hook

QA (来自官网)

1. https://zh-hans.reactjs.org/docs/hooks-intro.html#motivation
2. https://zh-hans.reactjs.org/docs/hooks-faq.html#are-hooks-slow-because-of-creating-functions-in-render

Hook 与 Fiber

```
type Fiber = {
  // 1. fiber节点自身状态相关
  pendingProps: any,
  memoizedProps: any,
  updateQueue: mixed,
  memoizedState: any,
  // 2. fiber节点副作用(Effect)相关
  flags: Flags,
  nextEffect: Fiber | null,
  firstEffect: Fiber | null,
  lastEffect: Fiber | null,
};
```

Hook 数据结构

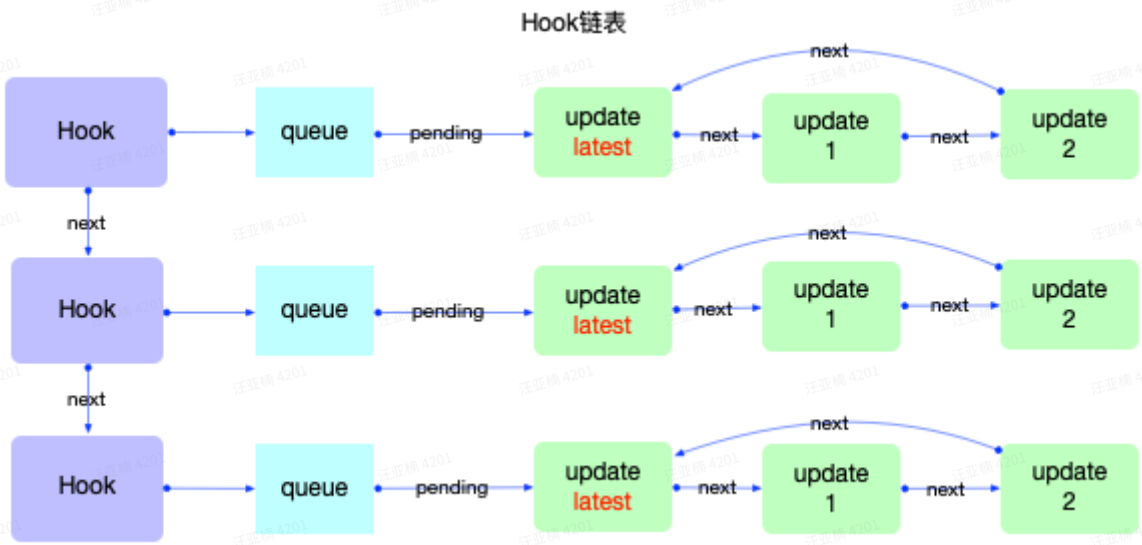
```
export type Hook = {
  memoizedState: any, // 当前状态
  baseState: any, // 基状态
  baseQueue: Update<any, any> | null, // 基队列
  queue: UpdateQueue<any, any> | null, // 更新队列
  next: Hook | null, // next指针
};

type UpdateQueue<S, A> = {
  pending: Update<S, A> | null,
  dispatch: (A => mixed) | null,
  lastRenderedReducer: ((S, A) => S) | null,
  lastRenderedState: S | null,
};
```

```
type Update<S, A> = {
  lane: Lane,
  action: A,
  eagerReducer: ((S, A) => S) | null,
  eagerState: S | null,
  next: Update<S, A>,
  priority?: ReactPriorityLevel,
};
```

从定义来看, `Hook` 对象共有 5 个属性

1. `hook.memoizedState` : 保持在内存中的局部状态.
2. `hook.baseState` : `hook.baseQueue` 中所有 `update` 对象合并之后的状态.
3. `hook.baseQueue` : 存储 `update对象` 的环形链表, 只包括高于本次渲染优先级的 `update对象`
4. `hook.queue` : 存储 `update对象` 的环形链表, 包括所有优先级的 `update对象` .
5. `hook.next` : `next` 指针, 指向链表中的下一个 `hook` .

所以 `Hook` 是一个链表, 单个 `Hook` 拥有自己的状态 `hook.memoizedState` 和自己的更新队列 `hook.queue`



Hook 分类

```
export type HookType =
  | 'useState'
  | 'useReducer'
  | 'useContext'
  | 'useRef'
  | 'useEffect'
  | 'useLayoutEffect'
```

```
  | 'useCallback'
  | 'useMemo'
  | 'useImperativeHandle'
  | 'useDebugValue'
  | 'useDeferredValue'
  | 'useTransition'
  | 'useMutableSource'
  | 'useOpaqueIdentifier';
```

状态 Hook

狭义上讲, `useState, useReducer` 可以在 `function组件` 添加内部的 `state`, 且 `useState` 实际上是 `useReducer` 的简易封装, 是一个最特殊(简单)的 `useReducer`. 所以将 `useState, useReducer` 称为 `状态Hook`.

广义上讲, 只要能实现数据持久化 `且没有副作用` 的 `Hook`, 均可以视为 `状态Hook`, 所以还包括 `useContext, useRef, useCallback, useMemo` 等. 这类 `Hook` 内部没有使用 `useState/useReducer`, 但是它们也能实现多次 `render` 时, 保持其初始值不变(即数据持久化)且没有任何 `副作用`

副作用 Hook

`状态Hook` 实现了状态持久化(等同于 `class组件` 维护 `fiber.memoizedState`), 那么 `副作用Hook` 则会修改 `fiber.flags`

在 `react` 内部, `useEffect` 就是最标准的 `副作用Hook`. 其他比如 `useLayoutEffect` 以及 `自定义Hook`, 如果要实现 `副作用`, 必须直接或间接的调用 `useEffect`

Hook全局变量

在分析 `renderWithHooks` 函数前, 有必要理解ReactFiberHooks头部定义的全局变量

```
// 渲染优先级
let renderLanes: Lanes = NoLanes;

// 当前正在构造的fiber, 等同于 workInProgress, 为了和当前hook区分, 所以将其改名
let currentlyRenderingFiber: Fiber = (null: any);

// Hooks被存储在fiber.memoizedState 链表上
let currentHook: Hook | null = null; // currentHook = fiber(current).memoizedState

```

```
let workInProgressHook: Hook | null = null; // workInProgressHook = fiber(workInProgress).memoizedState

// 在function的执行过程中，是否再次发起了更新. 只有function被完全执行之后才会重置.
// 当render异常时，通过该变量可以决定是否清除render过程中的更新.
let didScheduleRenderPhaseUpdate: boolean = false;

// 在本次function的执行过程中，是否再次发起了更新. 每一次调用function都会被重置
let didScheduleRenderPhaseUpdateDuringThisPass: boolean = false;

// 在本次function的执行过程中，重新发起更新的最大次数
const RE_RENDER_LIMIT = 25;
```

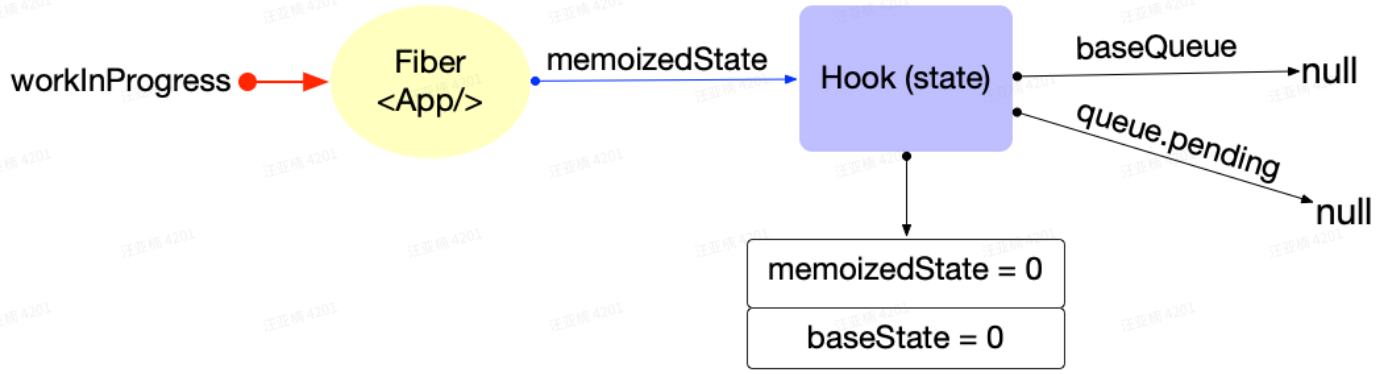
每个变量的解释, 可以对照源码中的英文注释, 其中最重要的有:

1. `currentlyRenderingFiber`: 当前正在构造的 fiber, 等同于 workInProgress
2. `currentHook 与 workInProgressHook`: 分别指向 `current.memoizedState` 和 `workInProgress.memoizedState`

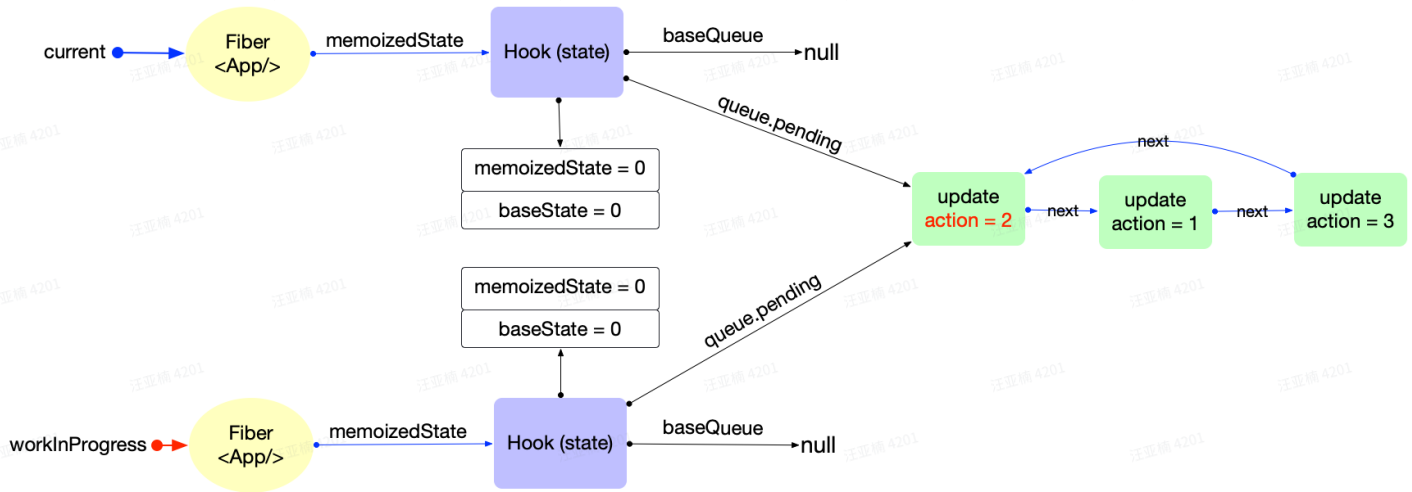
状态 Hook

```
import React, { useState } from 'react';
export default function App() {
  const [count, dispatch] = useState(0);
  return (
    <button
      onClick={() => {
        dispatch(1);
        dispatch(3);
        dispatch(2);
      }}
    >
      {count}
    </button>
  );
}
```

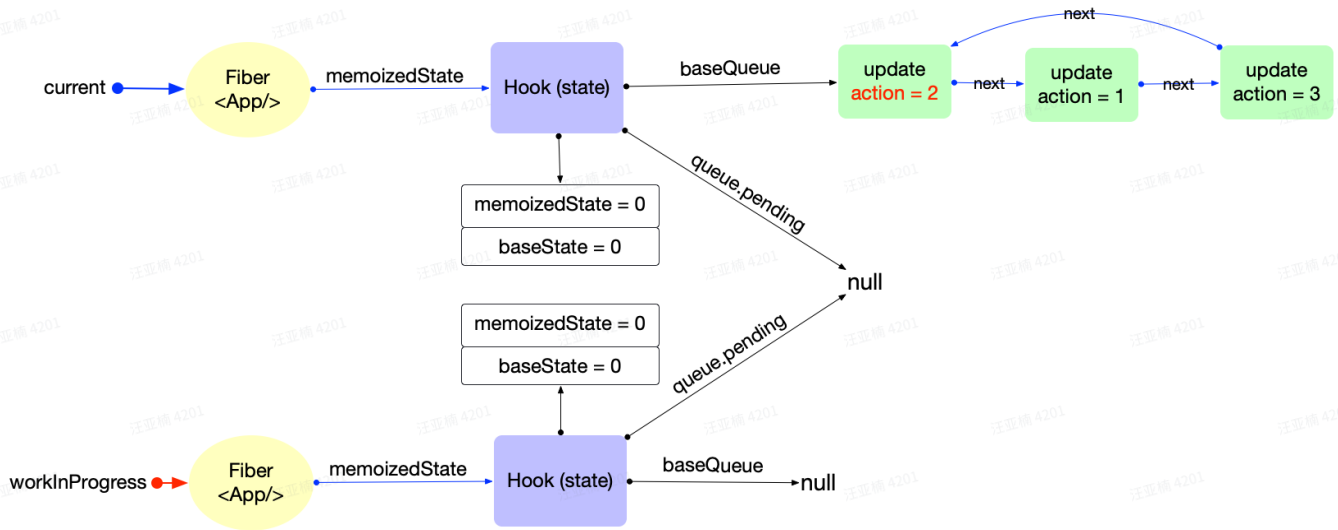
hook初始化

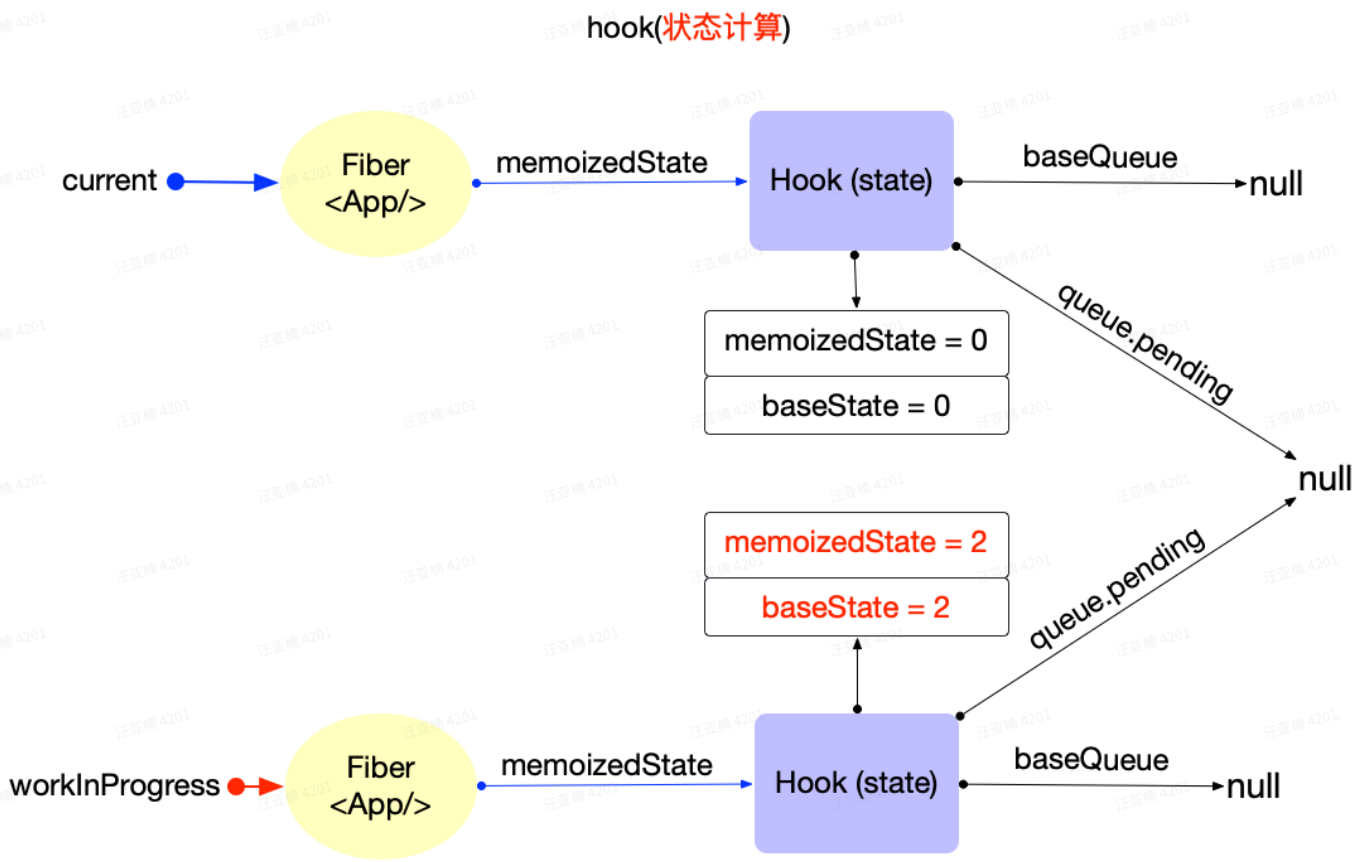


hook(baseQueue拼接前)



hook(baseQueue拼接后)





性能优化

`dispatchAction` 函数中, 在调用 `scheduleUpdateOnFiber` 之前, 针对 `update` 对象做了性能优化.

1. `queue.pending` 中只包含当前 `update` 时, 即当前 `update` 是 `queue.pending` 中的第一个 `update`
2. 直接调用 `queue.lastRenderedReducer` ,计算出 `update` 之后的 state, 记为 `eagerState`
3. 如果 `eagerState` 与 `currentState` 相同, 则直接退出, 不用发起调度更新.
4. 已经被挂载到 `queue.pending` 上的 `update` 会在下一次 `render` 时再次合并.

副作用 Hook

`useEffect` 与 `useLayoutEffect` 的区别如下:

1. `fiber.flags` 不同

- 使用 `useEffect` 时: `fiber.flags = UpdateEffect | PassiveEffect` .
- 使用 `useLayoutEffect` 时: `fiber.flags = UpdateEffect` .

2. `effect.tag` 不同

- 使用 `useEffect` 时: `effect.tag = HookHasEffect | HookPassive` .

- 使用 `useLayoutEffect` 时: `effect.tag = HookHasEffect | HookLayout`.

处理 Effect 回调

commitBeforeMutationEffects

第一阶段: dom 变更之前, 处理副作用队列中带有 `Passive` 标记的 `fiber` 节点

```
function commitBeforeMutationEffects() {
  while (nextEffect !== null) {
    // ...省略无关代码, 只保留Hook相关
    // 处理`Passive`标记
    const flags = nextEffect.flags;
    if ((flags & Passive) !== NoFlags) {
      if (!rootDoesHavePassiveEffects) {
        rootDoesHavePassiveEffects = true;
        scheduleCallback(NormalSchedulerPriority, () => {
          flushPassiveEffects();
          return null;
        });
      }
    }
    nextEffect = nextEffect.nextEffect;
  }
}
```

commitMutationEffects

第二阶段: dom 变更, 界面得到更新.

调用关系: `commitMutationEffects->commitWork->commitHookEffectListUnmount`.

- 注意在调用 `commitMutationEffects(HookLayout | HookHasEffect, finishedWork)` 时, 参数是 `HookLayout | HookHasEffect`, 所以只处理由 `useLayoutEffect()` 创建的 `effect`.
- 根据上文的分析 `HookLayout | HookHasEffect` 是通过 `useLayoutEffect` 创建的 `effect`. 所以 `commitMutationEffects` 函数只能处理由 `useLayoutEffect()` 创建的 `effect`.
- 同步调用 `effect.destroy()`.

commitLayoutEffects

第三阶段: dom 变更后

1. 调用关系: `commitLayoutEffects->commitLayoutEffectOnFiber(commitLifeCycles)->commitHookEffectListMount`

- 注意在调用 `commitHookEffectListMount(HookLayout | HookHasEffect, finishedWork)` 时, 参数是 `HookLayout | HookHasEffect` ,所以只处理由 `useLayoutEffect()` 创建的 `effect` .
- 调用 `effect.create()` 之后, 将返回值赋值到 `effect.destroy` .

组件销毁

当 `function` 组件被销毁时, `fiber` 节点必然会被打上 `Deletion` 标记, 即 `fiber.flags |= Deletion` . 带有 `Deletion` 标记的 `fiber` 在commitMutationEffects被处理

在 `commitDeletion` 函数之后, 继续调用 `unmountHostComponents->commitUnmount` , 在commitUnmount中, 执行 `effect.destroy()` , 结束整个闭环

useMemo与useCallback

mount

```
function mountMemo(nextCreate, deps) {
    var hook = mountWorkInProgressHook();
    var nextDeps = deps === undefined ? null : deps;
    var nextValue = nextCreate();
    hook.memoizedState = [nextValue, nextDeps];
    return nextValue;
}


function mountCallback(callback, deps) {
    var hook = mountWorkInProgressHook();
    var nextDeps = deps === undefined ? null : deps;
    hook.memoizedState = [callback, nextDeps];
    return callback;
}
```

update

```
function updateMemo(nextCreate, deps) {
```

```
    var hook = updateWorkInProgressHook();
    var nextDeps = deps === undefined ? null : deps;
    var prevState = hook.memoizedState;

    if (prevState !== null) {
      // Assume these are defined. If they're not, areHookInputsEqual will warn.
      if (nextDeps !== null) {
        var prevDeps = prevState[1];

        if (areHookInputsEqual(nextDeps, prevDeps)) {
          return prevState[0];
        }
      }
}

function updateCallback(callback, deps) {
    var hook = updateWorkInProgressHook();
    var nextDeps = deps === undefined ? null : deps;
    var prevState = hook.memoizedState;

    if (prevState !== null) {
      if (nextDeps !== null) {
        var prevDeps = prevState[1];

        if (areHookInputsEqual(nextDeps, prevDeps)) {
          return prevState[0];
        }
      }
}
```

useRef

```
function mountRef(initialValue) {
    var hook = mountWorkInProgressHook();
    var ref = {
      current: initialValue,
    };

    hook.memoizedState = ref;
    return ref;
 }

function updateRef(initialValue) {
```

```
    var hook = updateWorkInProgressHook();
    return hook.memoizedState;
}
```